

# Using REST APIs in the Frontend with JavaScript Fetch

---

A Complete Tutorial (CORS, Fetch API, Listing Data, Viewing Single Record, and Live Search)

---

## 1. Overview

This tutorial explains how to consume REST APIs in a frontend using the JavaScript **Fetch API**. It builds on previously created CRUD APIs (Create, Read, Update, Delete) and focuses on:

- Integrating APIs into the frontend
  - Enabling cross-origin communication using **CORS** in Express
  - Fetching and rendering list data in HTML
  - Fetching and displaying single record data in a modal
  - Implementing real-time search
  - Accessing uploaded images through Express static middleware
- 

## 2. Understanding API Consumption Methods

---

Available Methods for Calling APIs

Frontend applications can call backend APIs using several approaches:

1. **Native JavaScript Fetch API**
2. **jQuery AJAX (\$.ajax)**
3. **React.js frameworks (using fetch or axios)**
4. **AngularJS / Angular**
5. **Vue.js**
6. **Axios (3rd-party library)**

This tutorial focuses exclusively on the **Fetch API**.

---

## 3. Why CORS Is Required?

---

Same-Origin Policy Problem

If:

- Backend API runs at: <https://test.com>
- Frontend runs at: <https://x.com>

The browser **blocks cross-origin API calls** for security.

Solution: Enable CORS

CORS (Cross-Origin Resource Sharing) allows one origin to access resources from another origin.

When CORS Is Needed:

- Frontend domain and backend domain are **different**
- APIs must be accessible to external domains
- You need to allow specific origins, methods, and headers

---

## 4. Installing and Enabling CORS in Express.js

---

Step 1: Install CORS

```
npm install cors
```

Step 2: Require and Use as Middleware

```
const cors = require('cors');  
app.use(cors());
```

Place it **above all route definitions**, so all routes accept cross-origin requests.

---

## 5. Restricting CORS (Optional)

---

Allowing a Specific Domain

```
const corsOptions = {  
  origin: "https://example.com",  
  methods: ["GET", "POST"],  
  allowedHeaders: ["Content-Type", "Authorization"],  
  optionsSuccessStatus: 200  
};  
  
app.use(cors(corsOptions));
```

Allowing Multiple Origins

```
origin: ["https://example1.com", "https://example2.com"]
```

Enabling Credential-Based Access

```
credentials: true
```

---

## 6. Serving Uploaded Images (Static Middleware)

---

To allow frontend access to uploaded images:

```
const path = require("path");

app.use('/uploads', express.static(path.join(__dirname, 'uploads')));
```

---

## 7. Frontend Structure (HTML + Twig Example)

---

Sample HTML Table Structure

```
<table class="table">
  <thead>
    <tr>
      <th>Photo</th>
      <th>First Name</th>
      <th>Last Name</th>
      <th>Email</th>
      <th>Phone</th>
      <th>Gender</th>
      <th>Action</th>
    </tr>
  </thead>

  <tbody id="studentTableBody">
  </tbody>
</table>
```

---

## 8. Setting Up Fetch in the Frontend

---

Base API URL

```
const API_URL = "http://localhost:3000/api/students";
```

## 9. Fetching and Displaying All Students

---

### Step 1: Fetch Students

```
async function fetchStudents(search = "") {  
  const res = await fetch(`${API_URL}?search=${encodeURIComponent(search)}`);  
  const data = await res.json();  
  renderTable(data);  
}
```

### Step 2: Render Table Rows

```
function renderTable(students) {  
  const tbody = document.querySelector("#studentTableBody");  
  tbody.innerHTML = "";  
  
  students.forEach(student => {  
    tbody.innerHTML += `  
      <tr>  
        <td>  
            
        </td>  
        <td>${student.firstName}</td>  
        <td>${student.lastName}</td>  
        <td>${student.email}</td>  
        <td>${student.phone}</td>  
        <td>${student.gender}</td>  
        <td>  
          <button onclick="viewStudent('${student._id}')">View</button>  
          <button onclick="editStudent('${student._id}')">Edit</button>  
          <button onclick="deleteStudent('${student._id}')">Delete</button>  
        </td>  
      </tr>  
    `;  
  });  
}
```

### Step 3: Load Data on Page Load

```
fetchStudents();
```

---

## 10. Viewing a Single Student in Modal

---

## Step 1: Fetch Single Record

```
async function viewStudent(id) {  
  const res = await fetch(`${API_URL}/${id}`);  
  const student = await res.json();  
  
  showStudentModal(student);  
}
```

## Step 2: Populate and Display Modal

```
function showStudentModal(student) {  
  document.querySelector("#viewProfilePic").src =  
    `http://localhost:3000/uploads/${student.profilePic}`;  
  
  document.querySelector("#viewName").innerText =  
    `${student.firstName} ${student.lastName}`;  
  
  document.querySelector("#viewEmail").innerText = student.email;  
  document.querySelector("#viewPhone").innerText = student.phone;  
  document.querySelector("#viewGender").innerText = student.gender;  
  
  new bootstrap.Modal(document.querySelector('#viewStudentModal')).show();  
}
```

---

# 11. Search by Name (Live Search)

---

## Backend Change (Express Route)

Supports query string search:

```
GET /api/students?search=salman
```

Uses MongoDB `$or` + regex for case-insensitive search:

```
const search = req.query.search || "";  
  
const query = {  
  $or: [  
    { firstName: { $regex: search, $options: "i" } },  
    { lastName: { $regex: search, $options: "i" } }  
  ]  
};
```

```
const students = await Student.find(query);
```

---

### Frontend Live Search Logic

```
document.querySelector("#searchInput")
  .addEventListener("input", (e) => {
    fetchStudents(e.target.value);
  });
```

---

## 12. Refresh Button Functionality

```
function refreshData() {
  fetchStudents();
}
```

Bind to button:

```
<button onclick="refreshData()">Refresh</button>
```

---

## 13. Summary of Key Learning Points

- **CORS** must be enabled when frontend & backend run on different origins.
  - Use **Fetch API** for GET requests to retrieve data.
  - Render list data dynamically using DOM manipulation.
  - Use **Bootstrap modal** to display single student details.
  - Implement search using **query strings** and MongoDB **regex**.
  - Use `express.static()` to serve user-uploaded images.
-